34. Test Driven Development (TDD) + AI Döngüsü: Red-Green-Refactor

Yapay Zekanın Halüsinasyon Görmesini Engelleyen Nihai Geliştirme Ritmi
Bölüm 6 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Yapay zeka çağında TDD (Test-Driven Development) öldü mü, yoksa tam aksine AI'ı kontrol altında tutmanın tek bilimsel yolu haline mi geldi?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 TDD öldü söylemi yanlıştır; AI ile kod yazarken önce testi yazdırmak, AI'ın sınırlarını çizen en katı şartnamedir. - 📌 Red-Green-Refactor Döngüsü: 1. Başarısız test yaz (Kırmızı), 2. AI'a testi geçecek minimum kodu yazdır (Yeşil), 3. Kodu temizle (Refactor). - 📌 Bu döngüde yapay zeka sadece bir uygulayıcıdır; mimari kısıtları ve doğrulama kriterlerini belirleyen mühendistir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir okçuluk yarışmasını hayal edin.

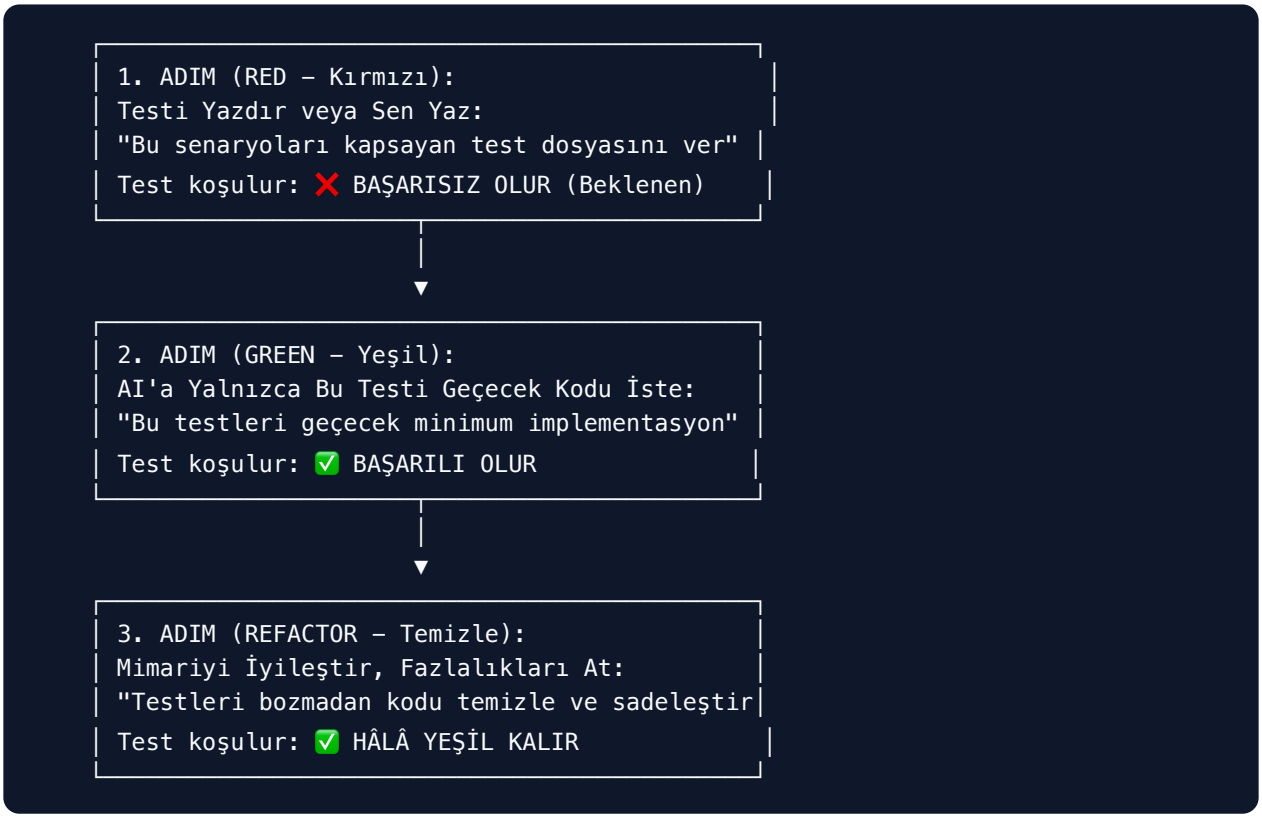
Geleneksel kötü yöntem: Oku rastgele ormana doğru fırlatırsınız. Okun düştüğü ağacın yanına gidip etrafına kırmızı bir hedef tahtası çizersiniz! *"Bakın tam hedeften vurdum!"* dersiniz. Komik, değil mi? Test yazmadan AI'a kod yazdırmak tam olarak budur.

TDD (Test-Driven Development) ise gerçek okçuluktur: 1. Önce 100 metre ileriye hedef tahtasını dikersiniz: **RED (Kırmızı)**. Henüz ok atılmadı, hedef boş. 2. Ardından oku tam merkeze fırlatırsınız: **GREEN (Yeşil)**. Test başarıyla geçti! 3. Duruşunuzu ve fiyonkları düzeltirsiniz: **REFACTOR (İyileştirme)**.

Yapay zekaya önce hedef tahtasını (testi) göstermezseniz, okun nereye gideceğini asla tahmin edemezsiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

AI Destekli TDD Döngüsü:



🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: TDD ve Temiz Kod Koçu
Bir 'İndirim Kuponu Doğrulama ve Uygulama' motoru geliştireceğiz.

## ADIM 1: RED (ÖNCE TEST)
Lütfen bana fonksiyonun kendisini YAZMA.
Sadece aşağıdaki gereksinimleri test eden Jest test dosyasını oluştur:
1. Kupon tarihi geçmişse `ExpiredCouponError` fırlatmalı.
2. Sepet tutarı minimum sepet limitinin altındaysa `MinimumSpendNotMetError`
fırlatmalı.
3. Yüzdelik indirimlerde (%20 indirim) tutarı doğru düşmeli.
4. Maksimum indirim tavanı varsa (örn: max 100 TL) indirimi o tutarla
sınırlandırmalı.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Geleneksel Hatalı Akış: Önce karmaşık kod yazıp sonra test yazmaya üşenmek
// Sonuç: Testi olmayan, sınır durumları belirsiz, refactor edilemeyen kod!
export function applyCouponBlind(cart: any, coupon: any) {
  return cart.total - coupon.amount;
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
// TDD Yeşil Adımı: Testleri geçen temiz ve sağlam implementasyon
export interface Cart {
  total: number;
}

export interface Coupon {
  code: string;
  discountPercent: number;
  maxDiscountAmount: number;
  minSpend: number;
  expiresAt: Date;
}

export function applyPromotionalCoupon(cart: Cart, coupon: Coupon): number {
  if (new Date() > coupon.expiresAt) {
    throw new Error("ExpiredCouponError: Kuponun süresi dolmuş.");
  }

  if (cart.total < coupon.minSpend) {
    throw new Error("MinimumSpendNotMetError: Minimum sepet tutarı karşılanmadı.");
  }

  const calculatedDiscount = (cart.total * coupon.discountPercent) / 100;
  const actualDiscount = Math.min(calculatedDiscount, coupon.maxDiscountAmount);

  return Math.round((cart.total - actualDiscount) * 100) / 100;
}
```

💡 **Kod Analizi:** TDD yaklaşımında önce test yazıldığı için kupon tavan sınırı (`Math.min`), tarih kontrolü ve minimum tutar kuralı koda daha baştan defansif olarak entegre edilmiş olur.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** AI'a hem testi hem kodu aynı anda yazdırıp testin çalıştığını doğrulamamak.
  - **Çözüm:** Önce testi yazdırın, testin kırmızı yandığını (hata verdiğini) gözlerinizle görün, ardından kodu yazdırın.
- **Tuzak:** Refactor aşamasını atlayıp 'nasılsa yeşil yandı' diyerek çirkin kodu öylece bırakmak.

  - **Çözüm:** Yeşil yandıktan sonra kodu temizleyin; testleriniz yeşil kaldığı sürece korkusuzca düzenleyin.
- **Tuzak:** Test senaryolarını çok geniş ve belirsiz tutmak.
  - **Çözüm:** Her test fonksiyonunda sadece tek bir iş kuralını (`it should...`) test edin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 34: AI ile TDD - Red, Green, Refactor Disiplini*

- 🖥️ **Slayt 1:** Neden TDD yapay zekayı evcilleştirmenin tek kesin yoludur?
- 🖥️ **Slayt 2:** 3 Adımlı Ritim: Red (Kırmızı), Green (Yeşil), Refactor (İyileştirme).
- 🖥️ **Slayt 3:** AI'a önce test yazdırmanın yarattığı kusursuz şartname etkisi.
- 🖥️ **Slayt 4:** Canlı Demo: 5 dakikada TDD ile kupon motoru inşa etmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Boş bir dosya açın. AI'a kupon testlerini yazdırın. Terminalde `npm test` çalıştırıp testin kırmızı yandığını (RED) gösterin. Ardından kodu üretip testin yeşile döndüğünü (GREEN) ve refactor sonrası yine yeşil kaldığını gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Geliştireceğiniz bir sonraki özellik için önce testi yazın veya AI'a sadece test dosyasını yazdırın. Testin başarısız olduğunu gördükten sonra kodu üretin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: TDD (Test-Driven Development) döngüsünün ilk adımı (RED) neden testin başarısız olduğunu görmeyi zorunlu kılar? - A) Geliştiricinin moralini bozmak için - B) Testin gerçekten çalıştığını, sahte (her zaman geçen) bir test olmadığını ve henüz yazılmamış özelliği doğru denetlediğini kanıtlamak için - C) Bilgisayarı test etmek için - D) Sadece C++ dilinde zorunlu olduğu için > **Doğru Yanıt:** B > **Açıklama:** Eğer test henüz kod yazılmadan önce yeşil yanıyorsa, o test hatalıdır veya hiçbir şey test etmiyordur. Kırmızı adımını görmek zorunludur.

Soru 2: TDD'nin üçüncü aşaması olan 'Refactor' adımında ne yapılır? - A) Proje silinir - B) Testlerin yeşil kalması güvencesi altında kod temizlenir, optimize edilir ve mimari olarak iyileştirilir - C) Yeni özellikler eklenir - D) Testler silinir > **Doğru Yanıt:** B > **Açıklama:** Refactor, çalışan sistemin dış davranışını değiştirmeden kod kalitesini, okunabilirliğini ve mimarisini iyileştirme aşamasıdır.